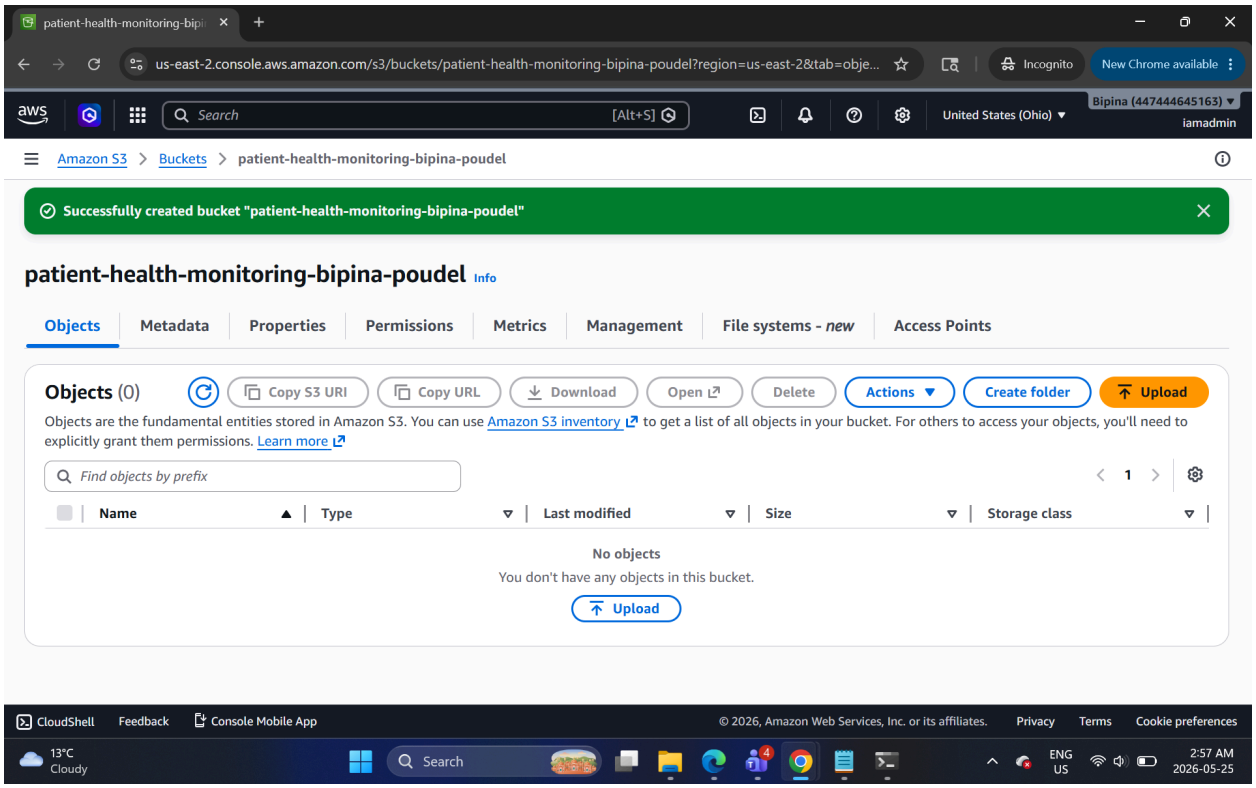
1. Project setup

Create an AWS project called **Patient Health Monitoring Data Pipeline**. The goal is to ingest, validate, clean, transform, and store patient monitoring data securely.

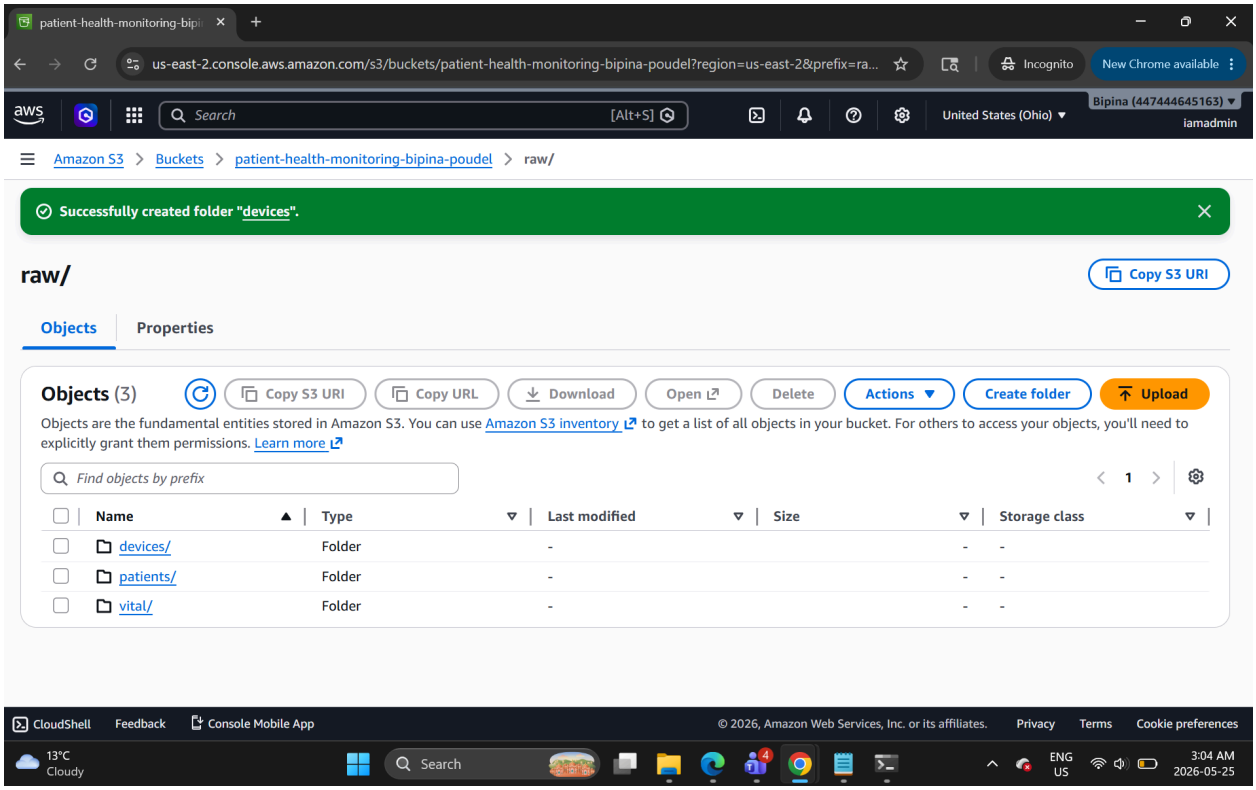
2. Create S3 bucket

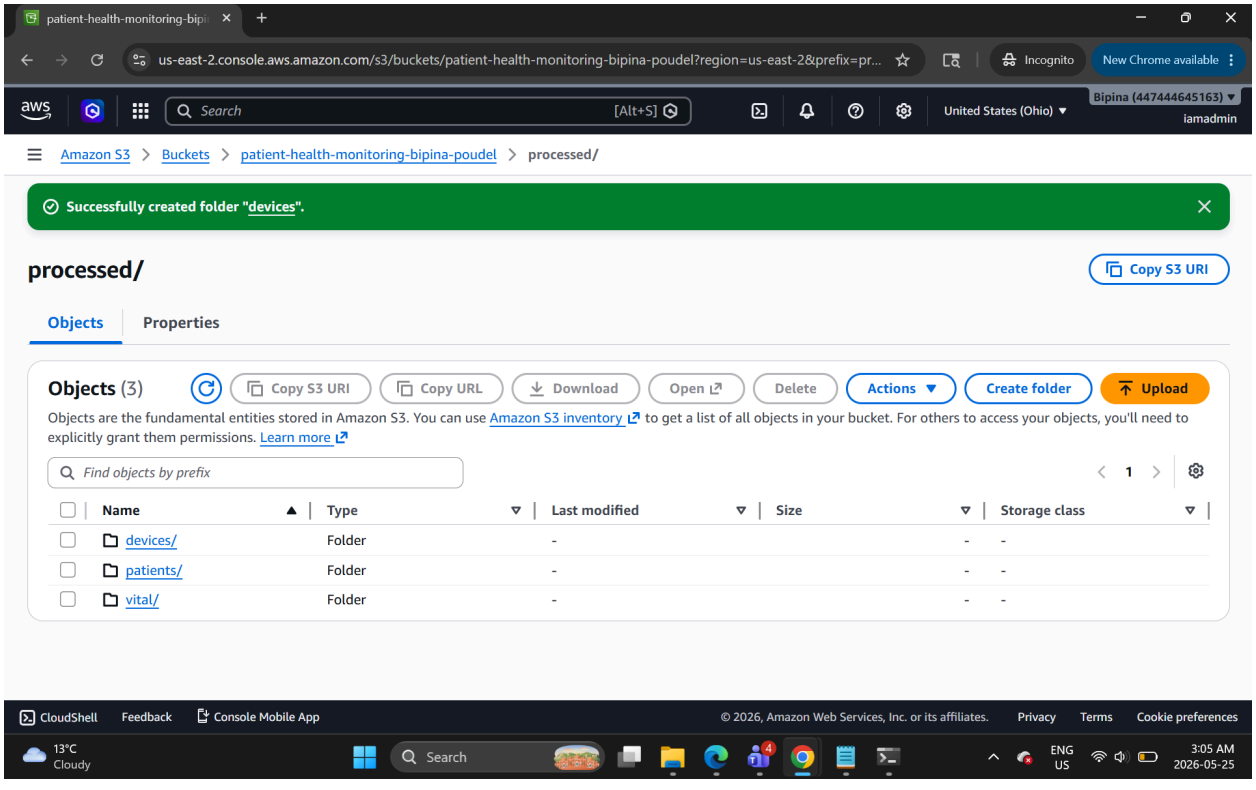
Created an S3 bucket: patient-health-monitoring-bipina-poudel

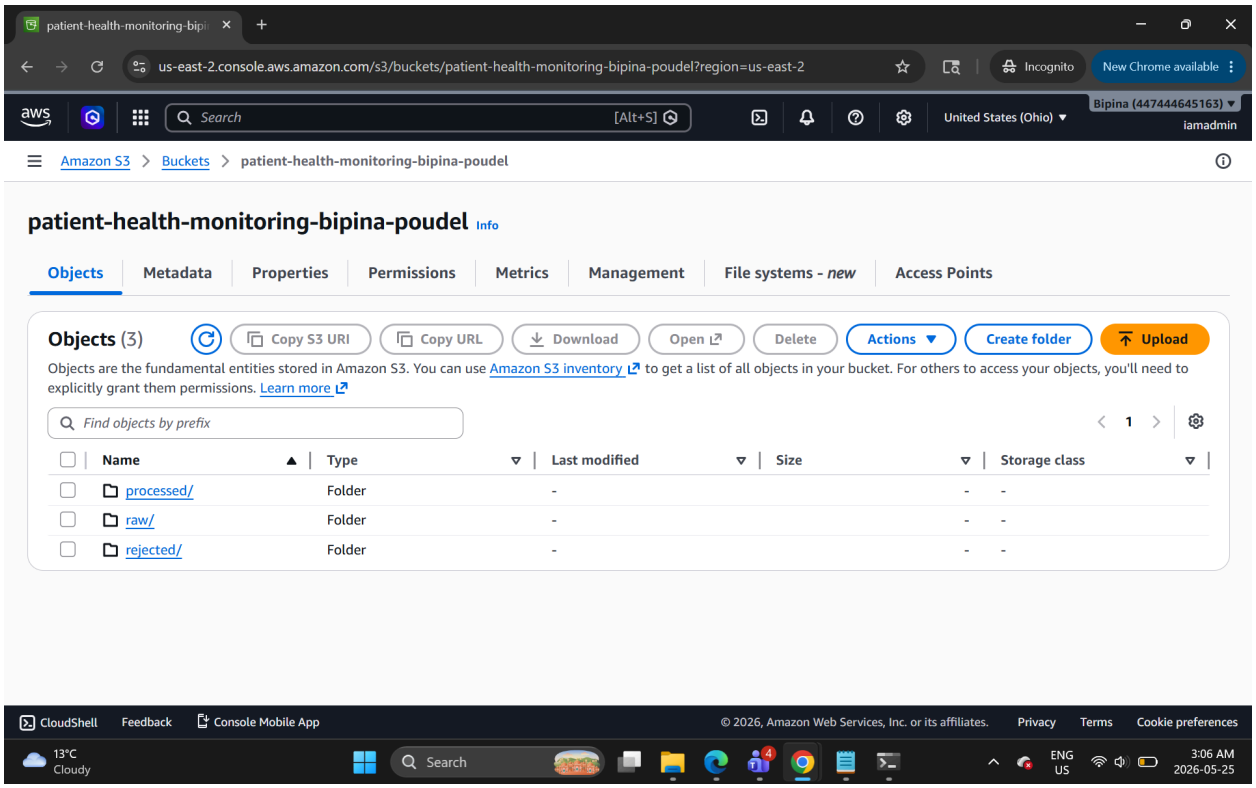


Inside it, created these folders:

raw/vitals/
raw/patients/
raw/devices/
processed/vitals/
processed/patients/
processed/reports/
rejected/

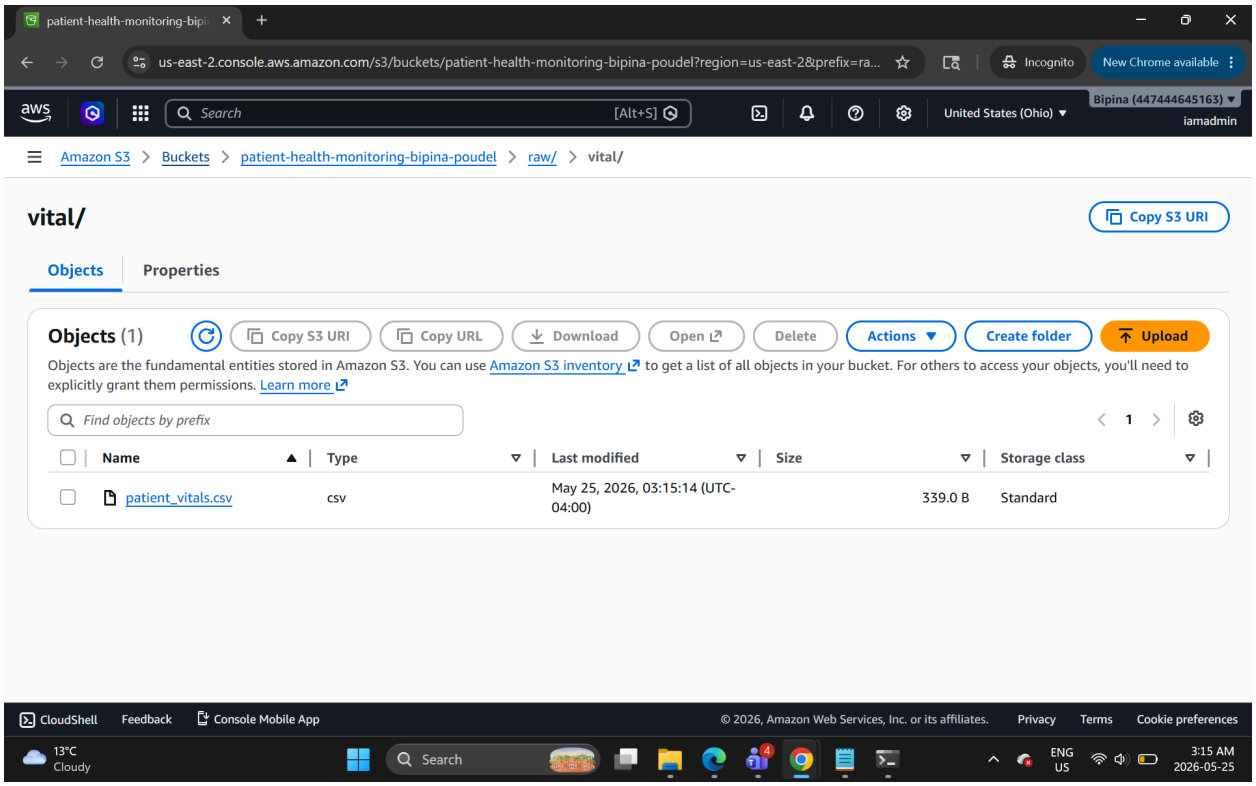

patient-health-monitoring-bipi
us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=ra...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > raw/
Successfully created folder "devices".
raw/
Copy S3 URI
Objects
Properties
Objects (3)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
devices/ Folder - - -
patients/ Folder - - -
vital/ Folder - - -
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Cloudy
Search
ENG
US
3:04 AM
2026-05-25

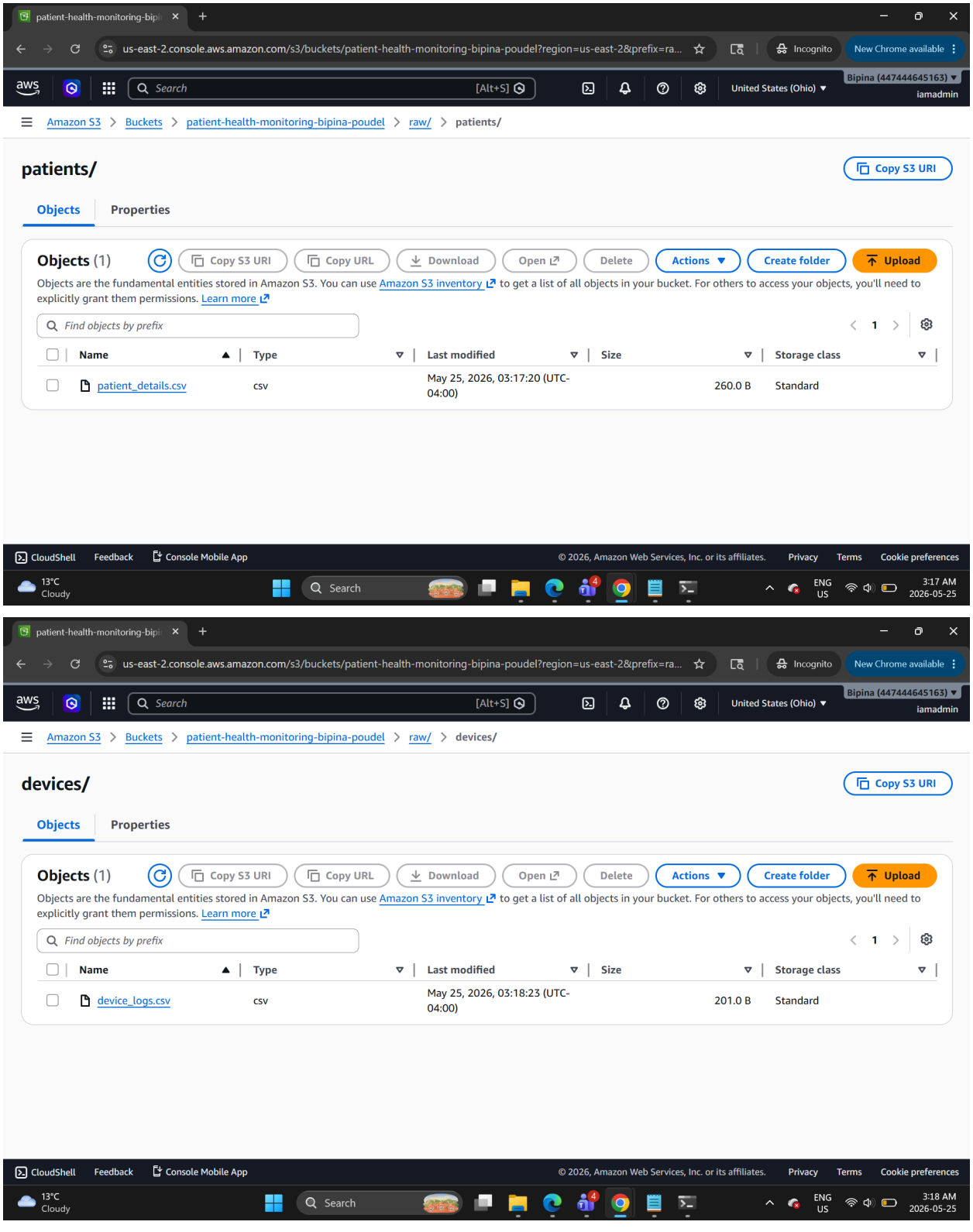

patient-health-monitoring-bipi
us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=pr...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > processed/
Successfully created folder "devices".
processed/
Copy S3 URI
Objects
Properties
Objects (3)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
devices/ Folder - - -
patients/ Folder - - -
vital/ Folder - - -
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Cloudy
Search
ENG
US
3:05 AM
2026-05-25



3. Created and uploaded CSV files

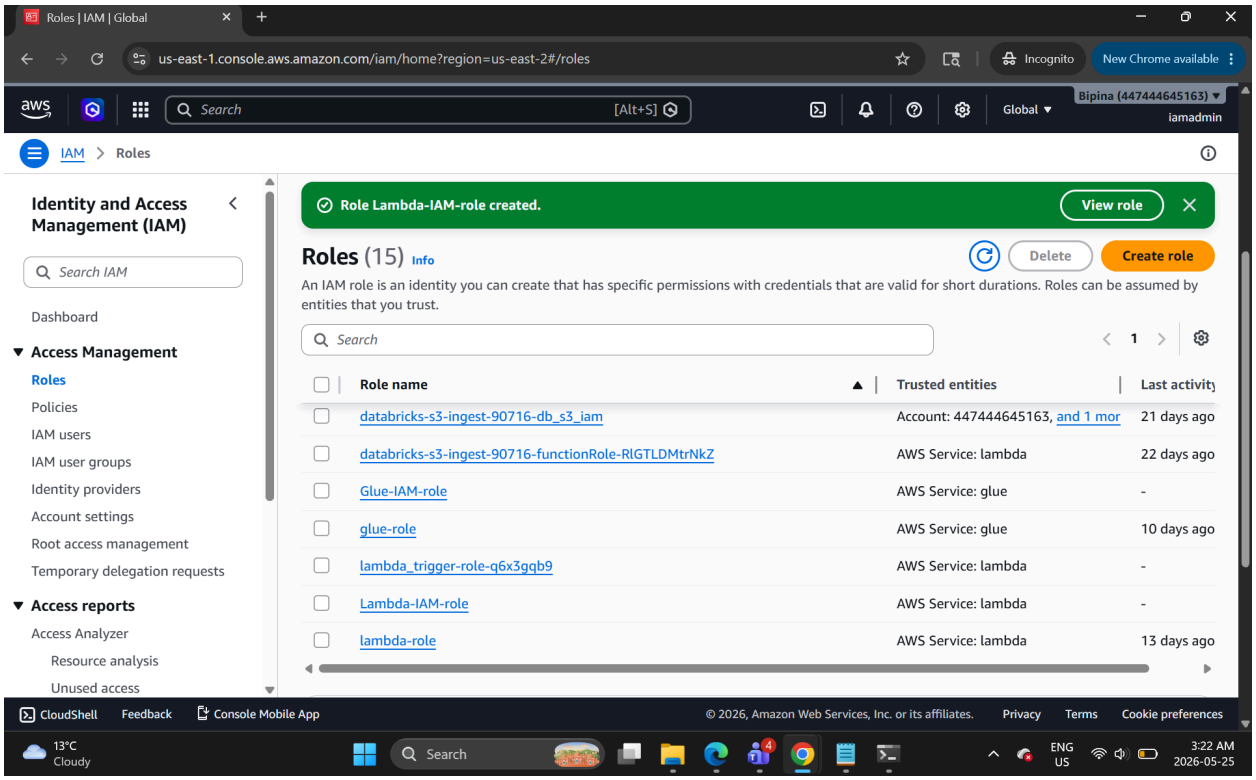
Created csv files and uploaded them in proper destinations.



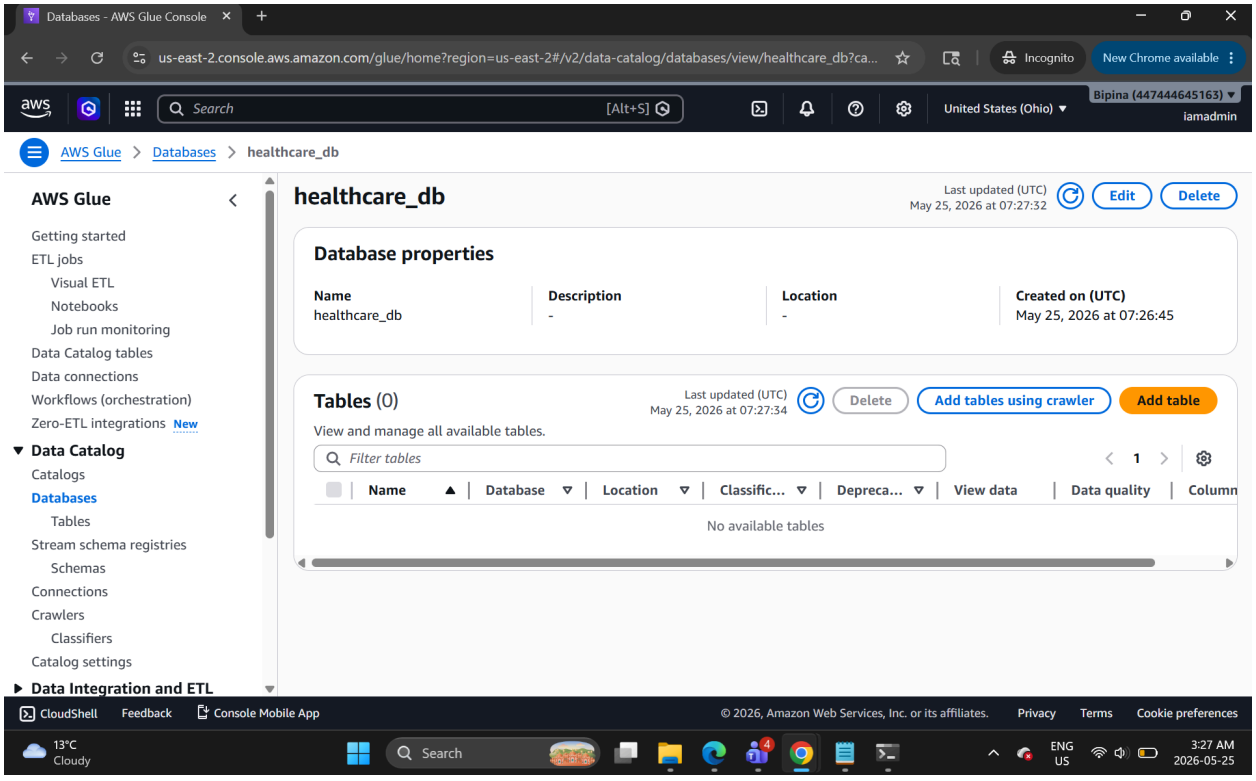

patient-health-monitoring-bipi
us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=ra...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > raw/ > patients/
patients/
Copy S3 URI
Objects
Properties
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
1
Name
Type
Last modified
Size
Storage class
patient_details.csv
csv
May 25, 2026, 03:17:20 (UTC-04:00)
260.0 B
Standard
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Cloudy
Search
ENG
US
3:17 AM
2026-05-25
patient-health-monitoring-bipi
us-east-2.console.aws.amazon.com/s3/buckets/patient-health-monitoring-bipina-poudel?region=us-east-2&prefix=ra...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon S3 > Buckets > patient-health-monitoring-bipina-poudel > raw/ > devices/
devices/
Copy S3 URI
Objects
Properties
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
1
Name
Type
Last modified
Size
Storage class
device_logs.csv
csv
May 25, 2026, 03:18:23 (UTC-04:00)
201.0 B
Standard
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Cloudy
Search
ENG
US
3:18 AM
2026-05-25

4. Created IAM roles

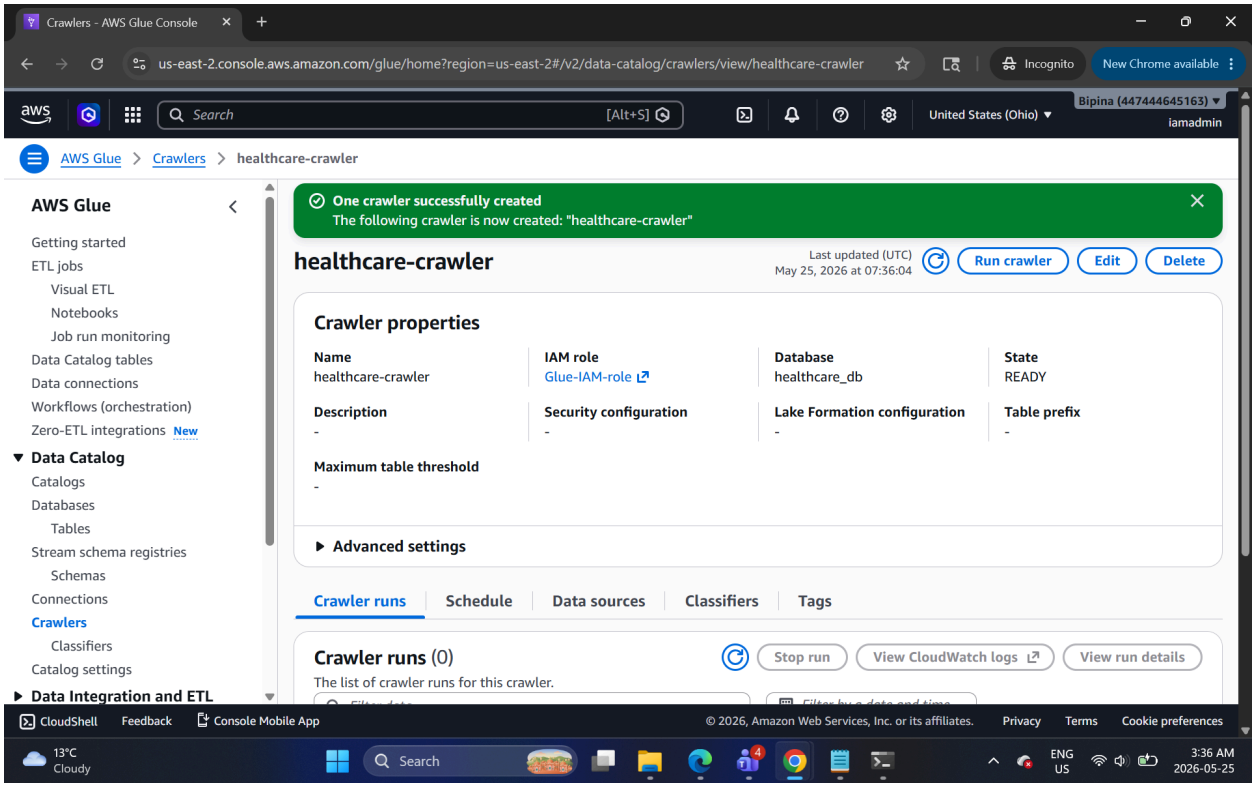
Glue IAM Role and Lambda IAM Role



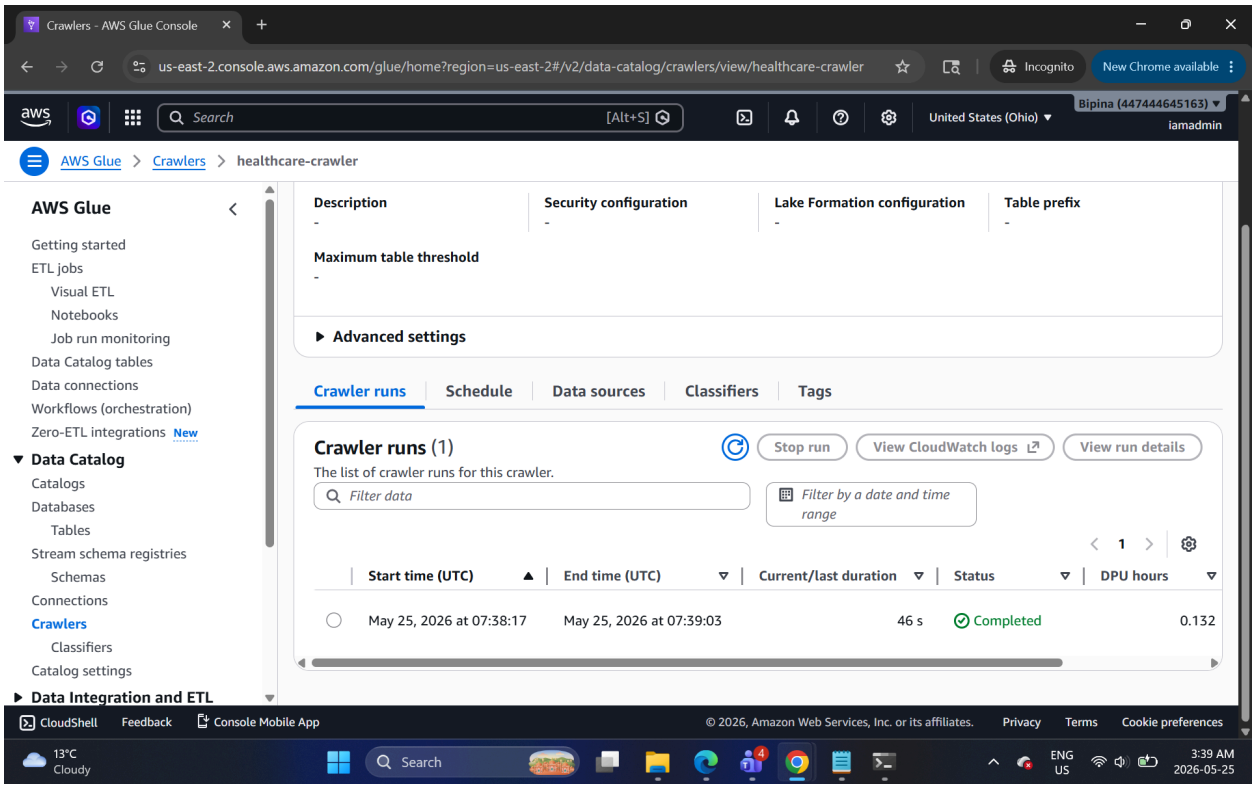
5. Created Glue database: healthcare_db



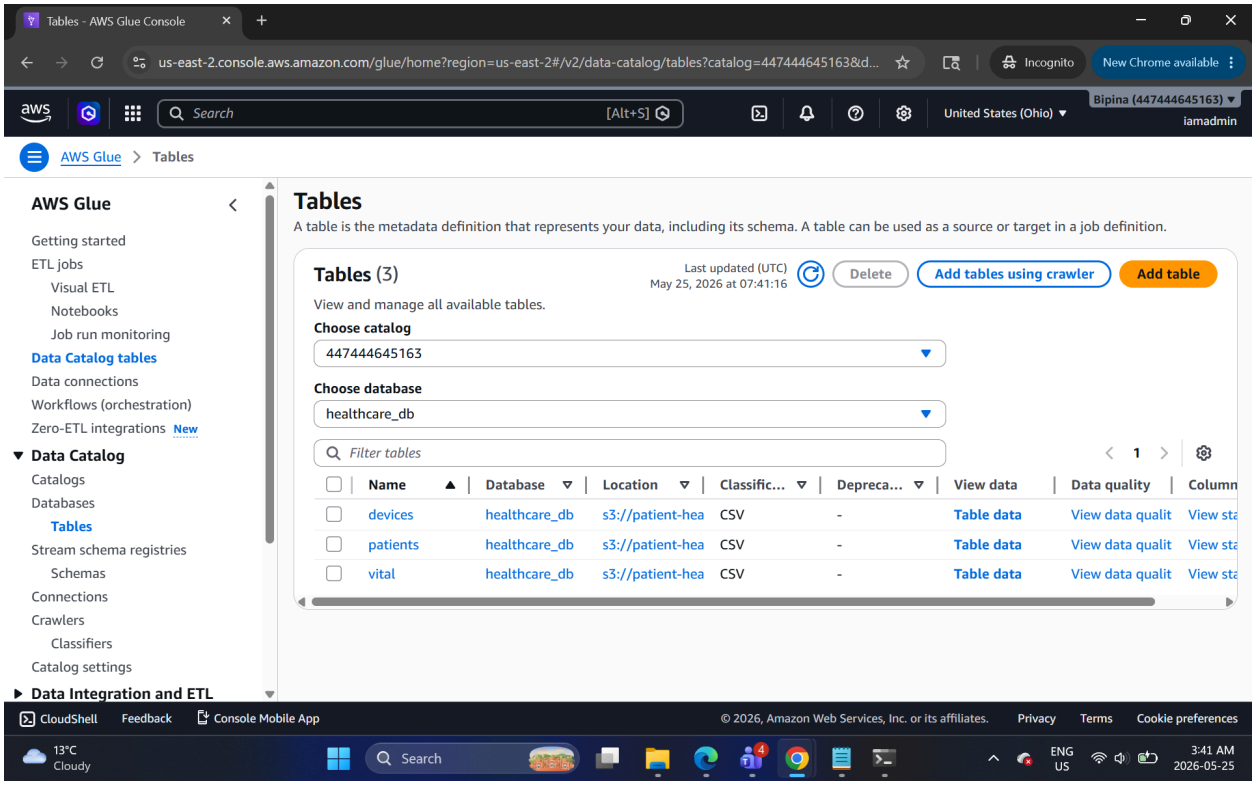
6. Created Glue crawler: healthcare-crawler



Started crawler successfully.



Crawler stores metadata in Glue Catalog successfully.



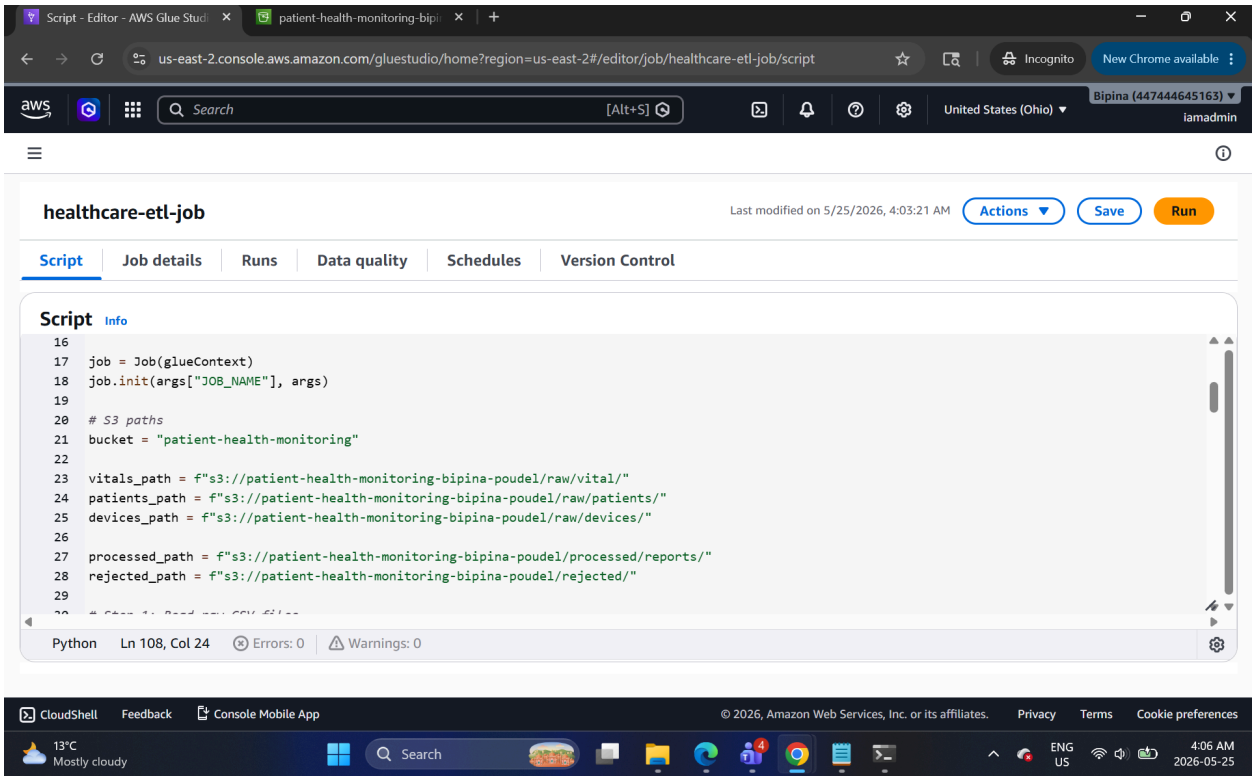
7. Created Glue ETL job

```
import sys
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from pyspark.sql.functions import (
    col, when, to_date, hour, lit, coalesce
)

# Glue job setup
args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args["JOB_NAME"], args)

# S3 paths
```

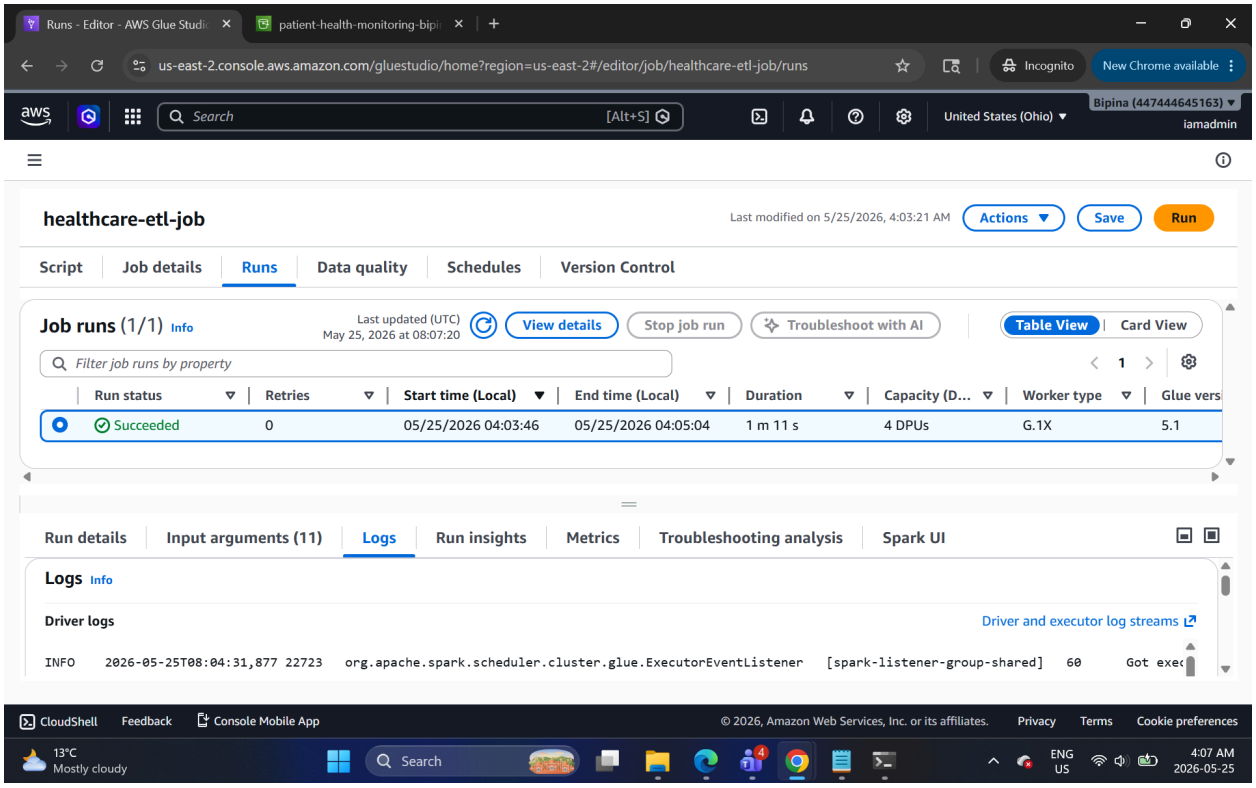
```
bucket = "patient-health-monitoring"

vitals_path = f"s3://patient-health-monitoring-bipina-poudel/raw/vital/"
patients_path = f"s3://patient-health-monitoring-bipina-poudel/raw/patients/"
devices_path = f"s3://patient-health-monitoring-bipina-poudel/raw/devices/"

processed_path = f"s3://patient-health-monitoring-bipina-poudel/processed/reports/"
rejected_path = f"s3://patient-health-monitoring-bipina-poudel/rejected/"

# Step 1: Read raw CSV files
vitals_df = spark.read.option("header", True).csv(vitals_path)
patients_df = spark.read.option("header", True).csv(patients_path)
devices_df = spark.read.option("header", True).csv(devices_path)

# Step 2: Convert data types
vitals_df = vitals_df.withColumn("heart_rate", col("heart_rate").cast("int")) \
    .withColumn("oxygen_level", col("oxygen_level").cast("int")) \
    .withColumn("temperature", col("temperature").cast("double")) \
    .withColumn("timestamp", col("timestamp").cast("timestamp"))

# Step 3: Remove duplicate records
vitals_df = vitals_df.dropDuplicates(["patient_id", "timestamp"])

# Step 4: Separate rejected/bad records
rejected_df = vitals_df.filter(
    col("patient_id").isNull() |
    col("heart_rate").isNull() |
    col("oxygen_level").isNull() |
    col("temperature").isNull()
)

# Write rejected records
rejected_df.write.mode("overwrite").option("header", True).csv(rejected_path)

# Keep valid records only
valid_vitals_df = vitals_df.filter(
    col("patient_id").isNotNull() &
    col("heart_rate").isNotNull() &
    col("oxygen_level").isNotNull() &
    col("temperature").isNotNull()
)

# Step 5: Create transformation columns
valid_vitals_df = valid_vitals_df.withColumn(
```

```
    "health_status",
    when(col("oxygen_level") < 90, "Critical")
    .when(col("temperature") > 100, "Fever")
    .when(col("heart_rate") > 100, "High Risk")
    .otherwise("Normal")
)

valid_vitals_df = valid_vitals_df.withColumn(
    "alert_flag",
    when(col("health_status") != "Normal", lit("Y")).otherwise(lit("N"))
)

valid_vitals_df = valid_vitals_df.withColumn(
    "monitoring_date",
    to_date(col("timestamp"))
)

valid_vitals_df = valid_vitals_df.withColumn(
    "monitoring_hour",
    hour(col("timestamp"))
)

# Step 6: Join patient vitals with patient details and device logs
final_df = valid_vitals_df.join(
    patients_df,
    on="patient_id",
    how="left"
).join(
    devices_df,
    on="patient_id",
    how="left"
)

# Step 7: Select final output columns
final_df = final_df.select(
    col("patient_id"),
    col("name").alias("patient_name"),
    col("disease"),
    col("heart_rate"),
    col("oxygen_level"),
    col("temperature"),
    col("health_status"),
    col("alert_flag"),
    col("device_type"),
```

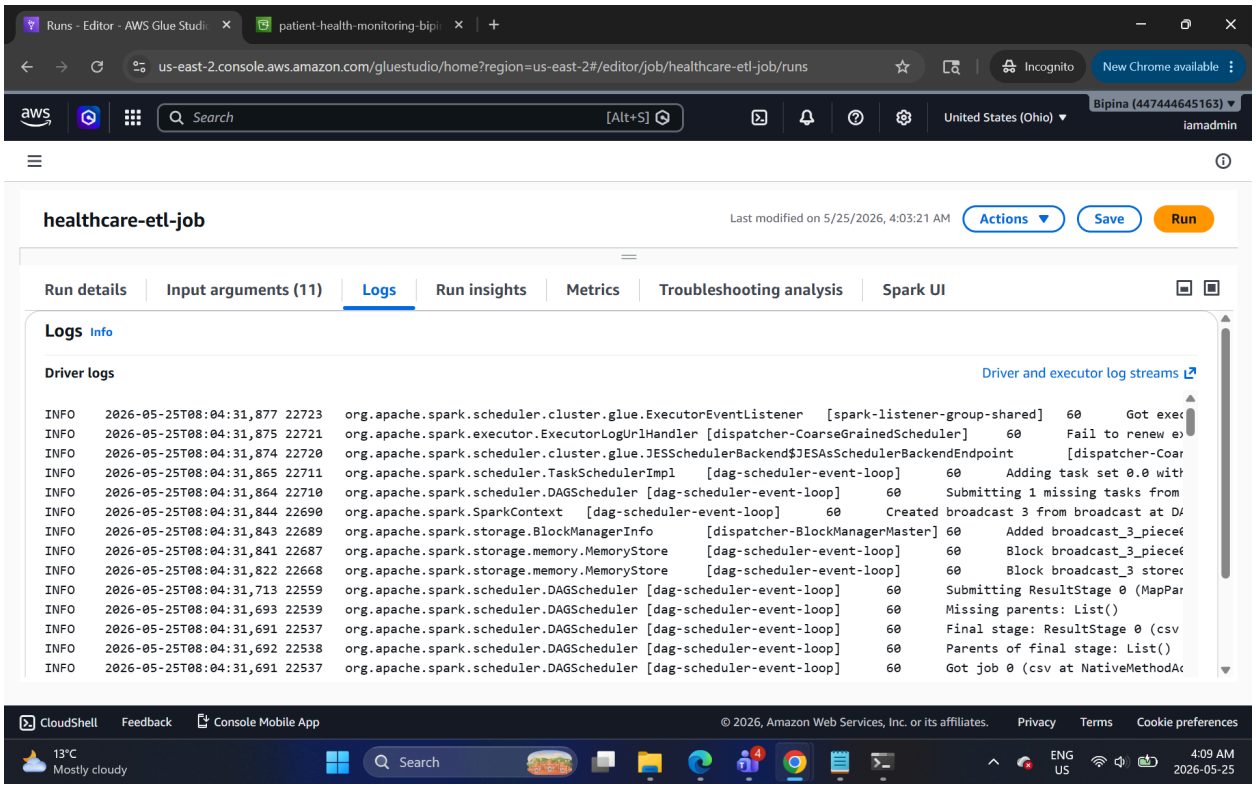
```
    col("monitoring_date"),
    col("monitoring_hour")
)

# Step 8: Write final processed data as Parquet
final_df.write.mode("overwrite").parquet(processed_path)

job.commit()
```



Run ETL job successfully.

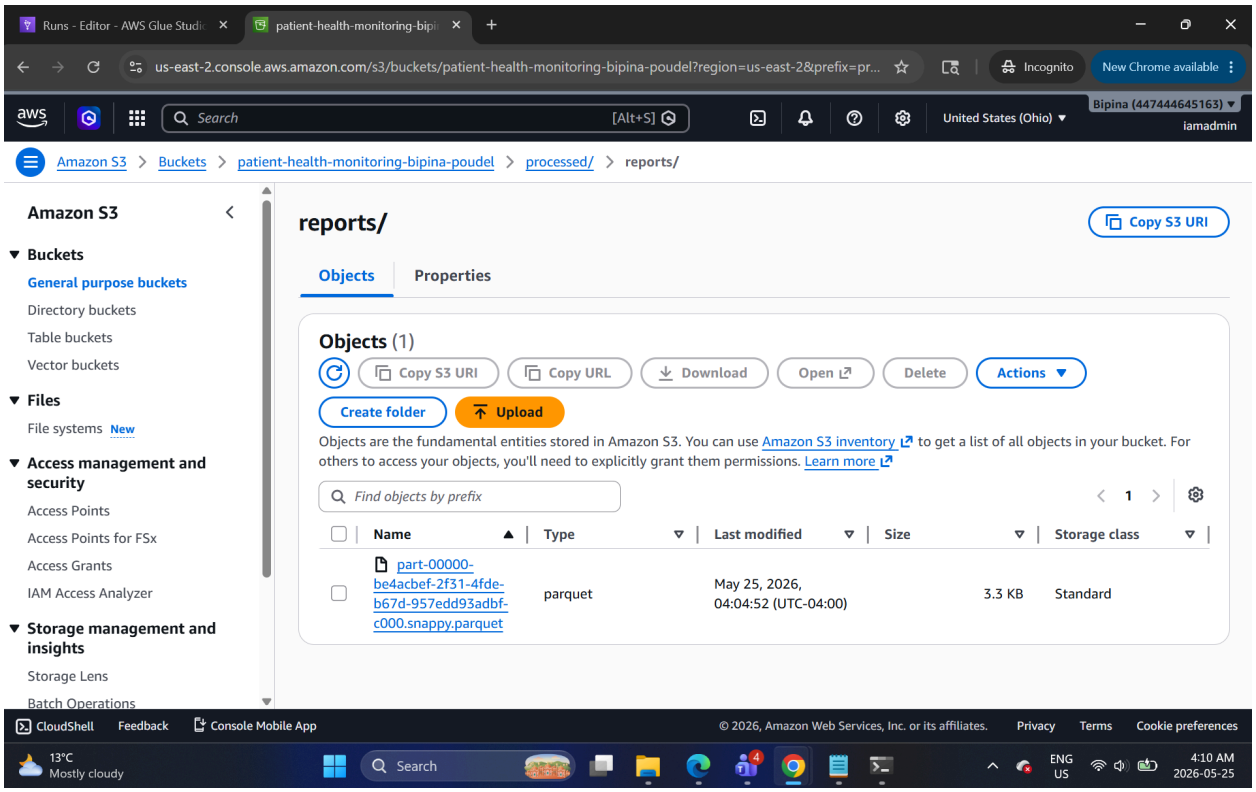


Logs

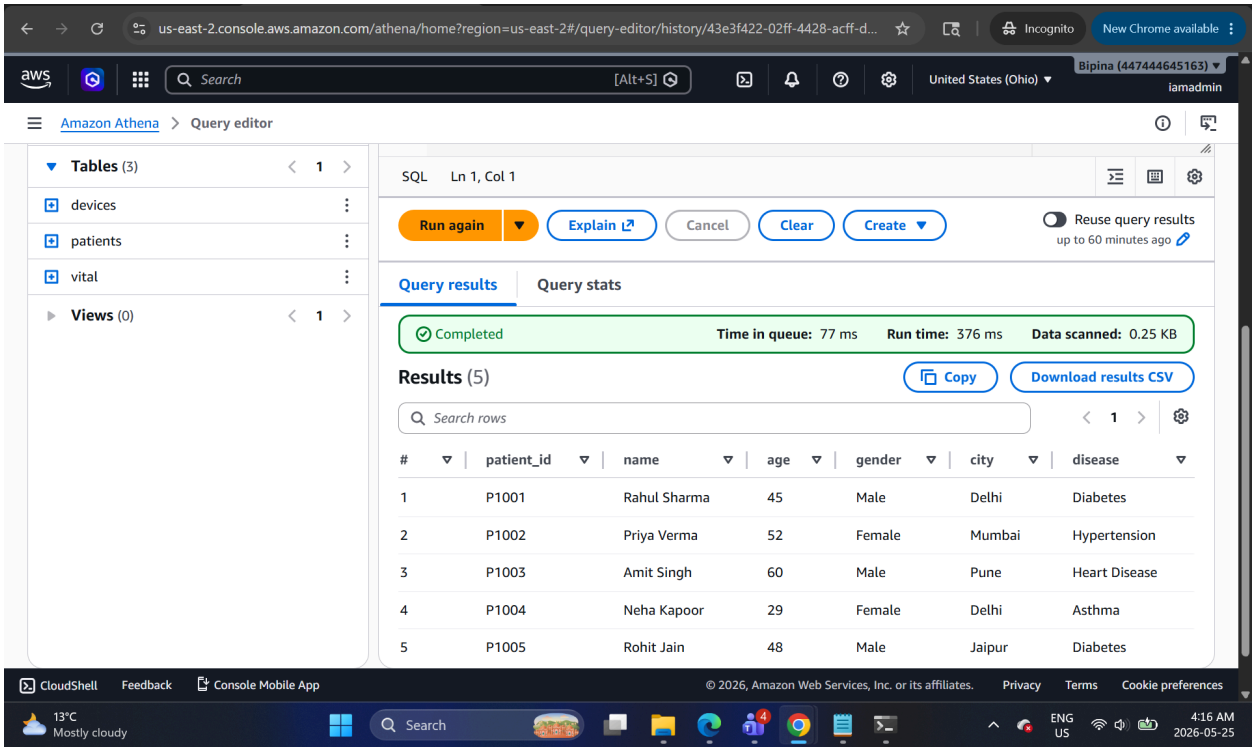


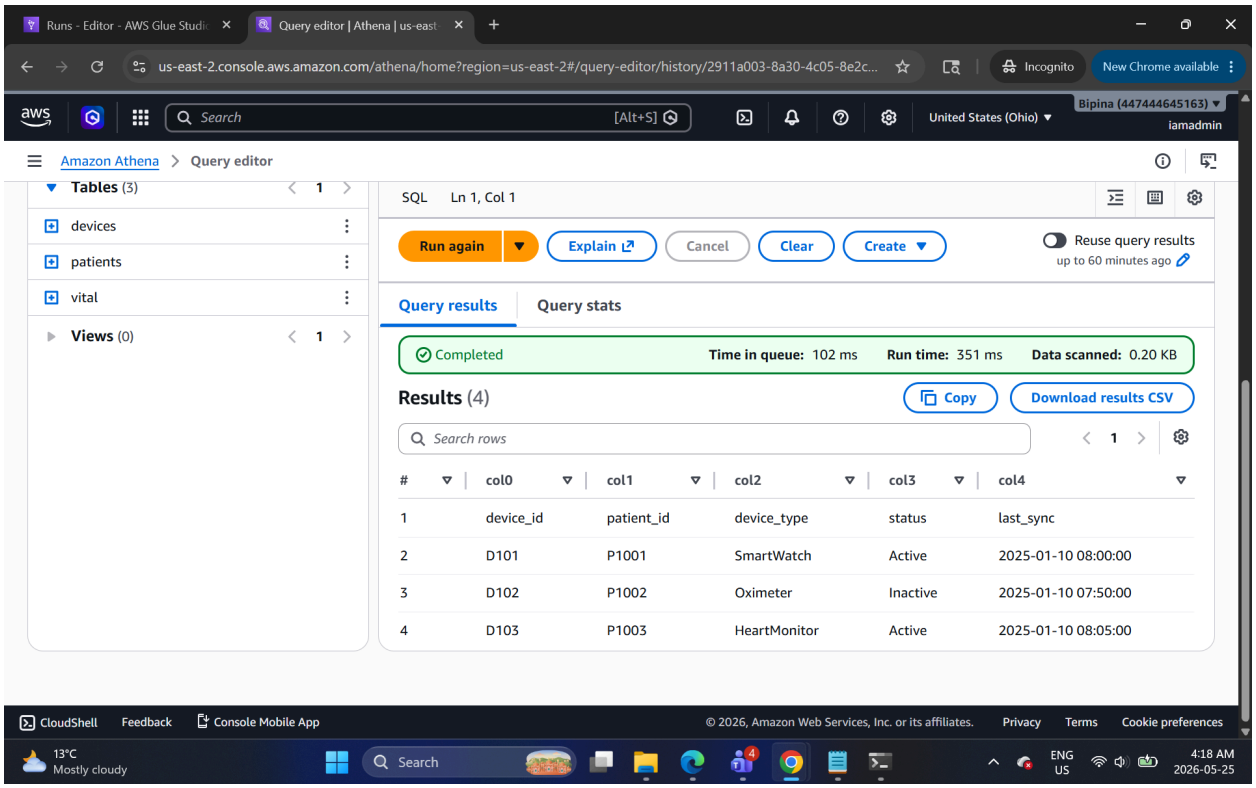
9. Checked final output

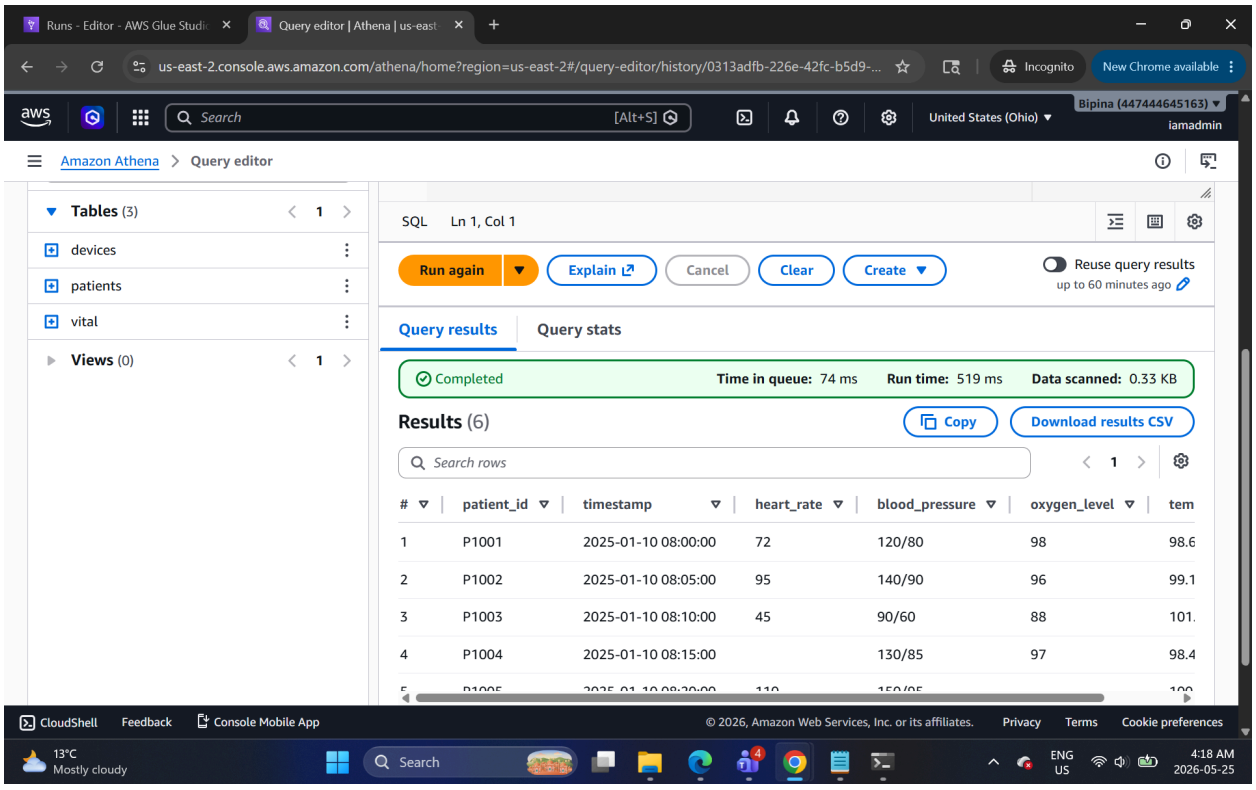
Verifying output Parquet File in S3



10. Athena Validation




Runs - Editor - AWS Glue Studi
Query editor | Athena | us-east-
us-east-2.console.aws.amazon.com/athena/home?region=us-east-2#/query-editor/history/2911a003-8a30-4c05-8e2c...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon Athena
Query editor
Tables (3)
devices
patients
vital
Views (0)
SQL
Ln 1, Col 1
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 102 ms
Run time: 351 ms
Data scanned: 0.20 KB
Results (4)
Copy
Download results CSV
Search rows
col0 col1 col2 col3 col4
1 device_id patient_id device_type status last_sync
2 D101 P1001 SmartWatch Active 2025-01-10 08:00:00
3 D102 P1002 Oximeter Inactive 2025-01-10 07:50:00
4 D103 P1003 HeartMonitor Active 2025-01-10 08:05:00
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Mostly cloudy
Search
ENG
US
4:18 AM
2026-05-25


Runs - Editor - AWS Glue Studi
Query editor | Athena | us-east-
us-east-2.console.aws.amazon.com/athena/home?region=us-east-2#/query-editor/history/0313adfb-226e-42fc-b5d9-...
Incognito
New Chrome available
Search
[Alt+S]
United States (Ohio)
Bipina (447444645163)
iamadmin
Amazon Athena
Query editor
Tables (3)
devices
patients
vital
Views (0)
SQL
Ln 1, Col 1
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 74 ms
Run time: 519 ms
Data scanned: 0.33 KB
Results (6)
Copy
Download results CSV
Search rows
patient_id timestamp heart_rate blood_pressure oxygen_level tem
1 P1001 2025-01-10 08:00:00 72 120/80 98 98.6
2 P1002 2025-01-10 08:05:00 95 140/90 96 99.1
3 P1003 2025-01-10 08:10:00 45 90/60 88 101.
4 P1004 2025-01-10 08:15:00 130/85 97 98.4
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
13°C
Mostly cloudy
Search
ENG
US
4:18 AM
2026-05-25